

Documentation technique

1. Réflexions initiales technologiques sur le sujet :

Avant de commencer l'ECF, j'ai pris un peu de temps pour réfléchir aux technologies que j'allais utiliser. Je voulais réaliser une application qui respecte les consignes de l'ECF et qui me permette de mettre en pratique ce que j'ai appris pendant ma formation.

Après réflexion, j'ai décidé d'utiliser Symfony en framework principal car c'est celui que j'ai étudié pendant la formation et Bootstrap pour le framework SCSS. Il permet de mieux organiser le projet, de gérer les utilisateurs, les formulaires et la sécurité plus facilement.

Pour la base de données, j'ai utilisé MySQL avec Doctrine ORM. Doctrine me permet de manipuler les données à partir des entités du projet sans écrire toutes les requêtes SQL manuellement.

J'ai aussi utilisé MongoDB pour le stockage des données utilisées par les statistiques. Ça m'a permis de découvrir les bases de données NoSQL et de comprendre dans quel cas elle peut être utilisée.

Pour la partie interface, j'ai utilisé Twig, Bootstrap, HTML, CSS et un peu de JavaScript pour créer un site responsive et agréable à utiliser sur ordinateur ainsi que sur mobile.

J'ai aussi choisi Docker pour préparer le déploiement de l'application. Cela permet d'avoir le même environnement en développement et sur le serveur, ce qui facilite le déploiement et les futures mises à jour.

Ces technologies répondaient aux besoins du projet et m'ont permis de développer une application complète tout en mettant en pratique les compétences acquises pendant ma formation.

2. Configuration de votre environnement de travail :

Pour développer le projet Vite & Gourmand, j'ai préparé un environnement de travail qui me permettait de développer, tester et déployer l'application.

Mon poste de développement fonctionne sous Windows 11. J'ai utilisé Visual Studio Code comme éditeur de code, car il est léger, simple à utiliser et possède de nombreuses extensions adaptées à Symfony.

Le projet a été créé avec Symfony et Composer. J'ai utilisé Git et GitHub pour sauvegarder le code source et travailler avec plusieurs branches afin de séparer les nouvelles fonctionnalités de la version stable.

Pour la base de données relationnelle, j'ai utilisé MySQL avec Doctrine ORM. Les statistiques sont enregistrées dans MongoDB.

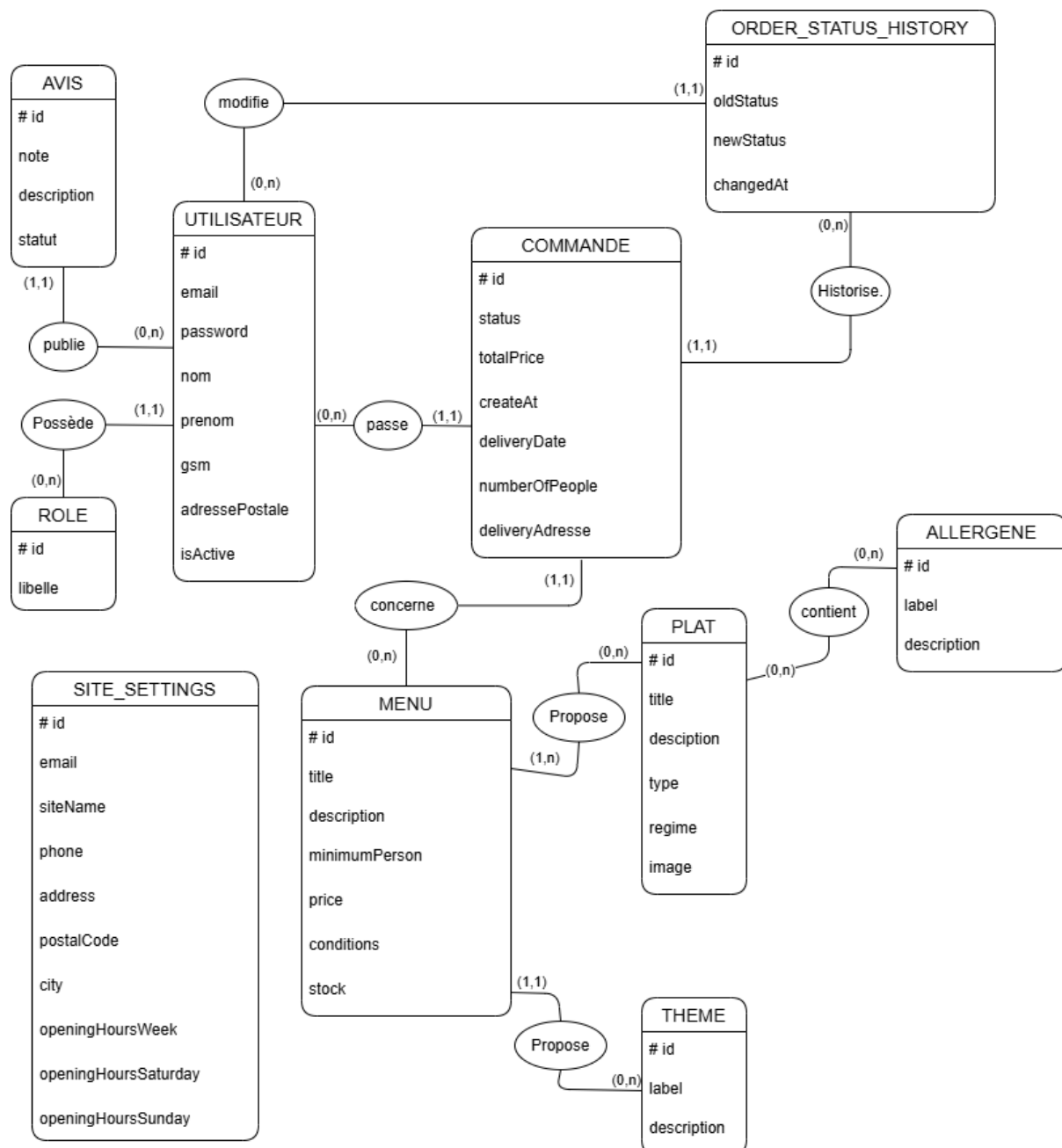
Pour le développement des envois de mails, j'ai utilisé Mailpit, il m'a permis de tester les envois sans utiliser une véritable adresse e-mail.

J'ai installé Docker Desktop avec WSL2 pour préparer le déploiement du projet sur un serveur VPS. Ça m'a permis d'utiliser le même environnement en local et sur le serveur.

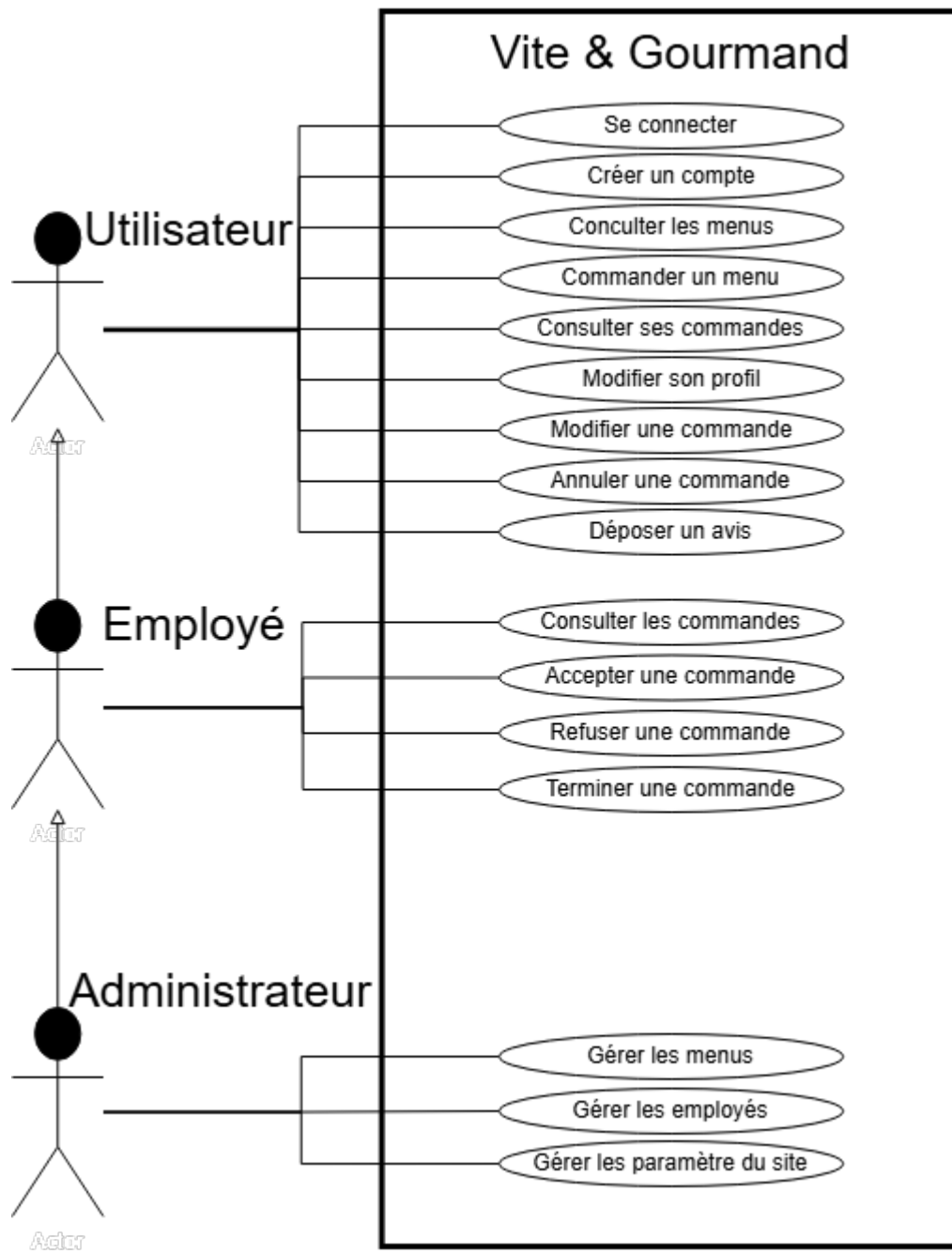
Toutes les étapes d'installation et des prérequis est détaillé dans le fichier README.md du projet.

3. Modèle conceptuel de données :

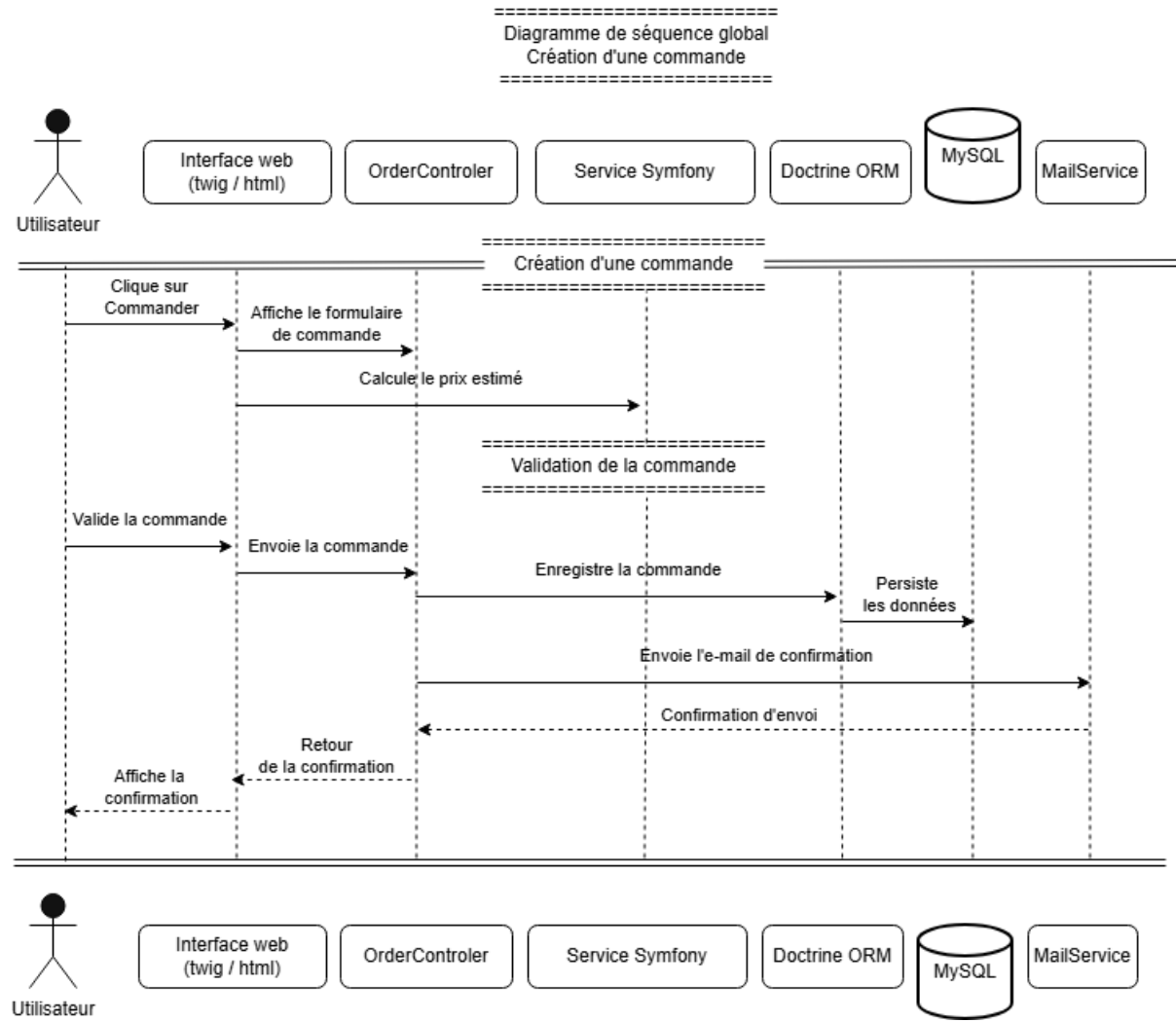
Le modèle conceptuel de données ci-dessous représente la structure de la base de données de l'application Vite & Gourmand. Il présente les principales entités, leurs attributs ainsi que les relations entre elles. Ce modèle a été adapté au cours du développement afin de refléter la version finale de l'application.



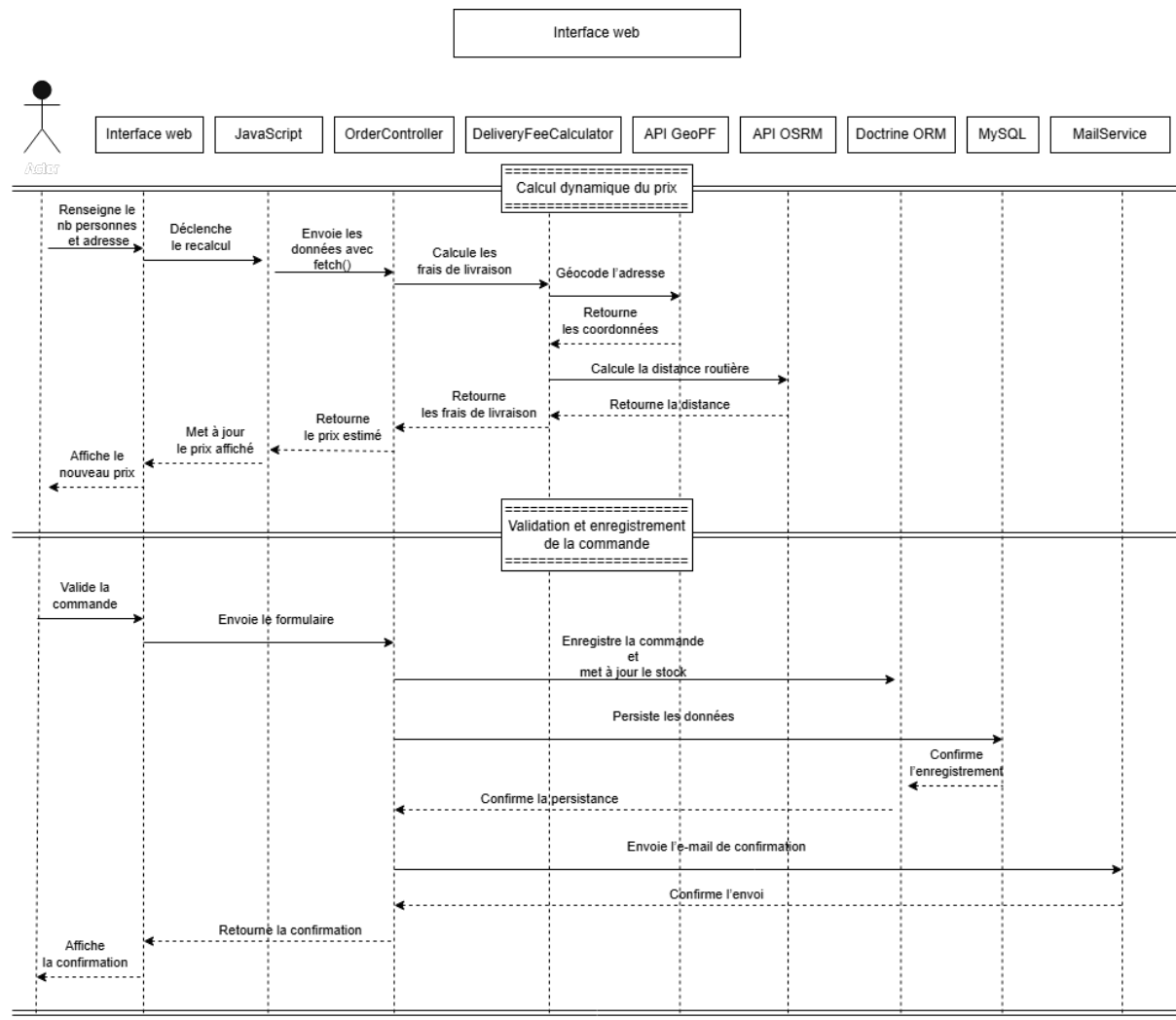
4. Diagramme de cas d'utilisation :



5. Diagramme de séquence :



6. Diagramme de séquence détaillé:



7. Déploiement de l'application :

7.1 Objectif du déploiement :

Pour le déploiement de mon application (Vite & Gourmand), j'ai choisi d'utiliser Docker. Le but était de pouvoir retrouver le même environnement de travail sur mon ordinateur et sur le serveur, sans avoir à tout réinstallé ou reconfiguré à chaque fois.

Cette solution me permet de regrouper tous les éléments nécessaires au fonctionnement de l'application dans des conteneurs séparés. Chaque service possède son propre environnement, ce qui évite les conflits entre les différentes technologies utilisées.

L'application repose sur plusieurs services :

- PHP pour exécuter Symfony ;
- MySQL pour la base de données principale ;
- MongoDB pour les statistiques ;
- Mailpit pour tester l'envoi des e-mails pendant le développement.

Grâce à Docker Compose, l'ensemble de ces services peut être démarré automatiquement avec une seule commande.

7.2 Pourquoi avoir choisi Docker ? :

Au début, je développais mon application directement sur mon pc. En avançant dans le projet, j'ai souhaité me rapprocher d'un environnement de production afin de faciliter le futur déploiement sur le serveur.

Docker m'a permis de créer un environnement identique entre mon pc de développement et le VPS utilisé pour les tests. Cela évite les différences de configuration qui peuvent provoquer des erreurs lors de la mise en ligne.

J'ai également choisi Docker car il facilite la maintenance du projet. Chaque service fonctionne dans son propre conteneur et peut être mis à jour ou redémarré indépendamment des autres.

Enfin, cette solution me permettra de réutiliser plus facilement cette méthode pour mes futurs projets.

7.3 Architecture du déploiement :

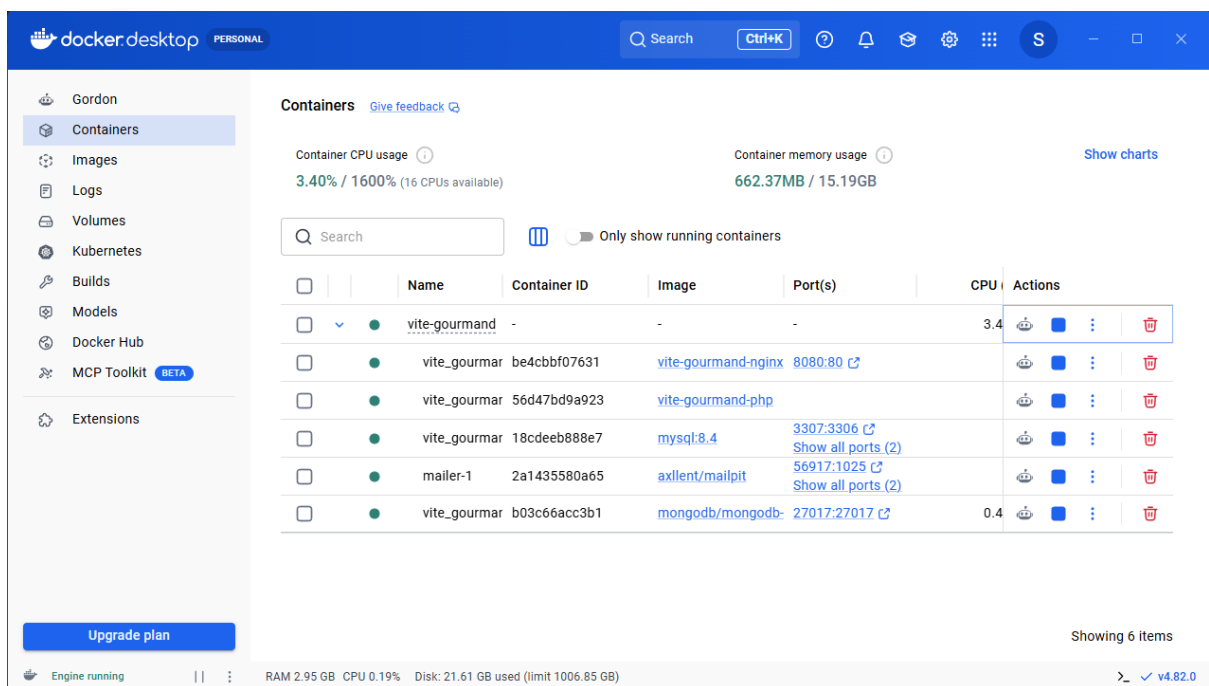
Pour déployer Vite et gourmand, j'ai utilisé plusieurs conteneurs Docker. J'ai séparé les différents services dans plusieurs conteneurs pour que chacun ait son propre rôle.

J'ai utilisé le fichier compose.yaml pour lancer tous les services en une seule commande.

Les conteneurs utilisés sont les suivants :

- Nginx : serveur web qui reçoit les requêtes des utilisateurs et les transmet à l'application Symfony.
- PHP : exécute l'application Symfony et traite les requêtes des utilisateurs.
- MySQL : stocke les données principales de l'application (utilisateurs, menus, commandes, horaires...).
- MongoDB : enregistre les données utilisées pour les statistiques du tableau de bord administrateur.
- Mailpit : permet de tester l'envoi des e-mails sans utiliser un véritable serveur SMTP.
-

Chaque conteneur communique avec les autres grâce au réseau Docker créé automatiquement par Docker Compose.



7.4 Mise en place de l'environnement :

Une fois Docker installé sur le poste de développement et sur le VPS, j'ai récupéré le projet puis construit les différents conteneurs grâce au fichier compose.yaml.

Après le démarrage des conteneurs, j'ai installé les dépendances du projet avec Composer puis créé la base de données à l'aide des migrations Doctrine.

Les différentes variables d'environnement ont ensuite été configurées afin de permettre la connexion entre Symfony, MySQL, MongoDB et Mailpit.

Une fois cette étape terminée, l'application était accessible et l'ensemble des services communiquaient correctement entre eux.

```
+ ] up 7/7
✓ Image vite-gourmand-php          Built
✓ Image vite-gourmand-nginx        Built
✓ Container vite_gourmand_mailpit  Running
✓ Container vite_gourmand_mysql    Healthy
✓ Container vite_gourmand_mongodb  Running
✓ Container vite_gourmand_php       Started
✓ Container vite_gourmand_nginx     Started
```

7.5 Commandes principales utilisées :

Au cours du déploiement, plusieurs commandes Docker et Symfony ont été utilisées.

Construction des images Docker

```
0e:~/projects/ecf---Vite-Gourmand/vite-gourmand$ docker compose build
```

Démarrage des conteneurs

```
0e:~/projects/ecf---Vite-Gourmand/vite-gourmand$ docker compose up -d
```

Vérification des conteneurs

```
0e:~/projects/ecf---Vite-Gourmand/vite-gourmand$ docker compose ps
```

Installation des dépendances Symfony

```
0e:~/projects/ecf---Vite-Gourmand/vite-gourmand$ docker compose exec php composer install
```

Exécution des migrations Doctrine

```
ecf---Vite-Gourmand/vite-gourmand$ docker compose exec php php bin/console doctrine:migrations:migrat
```

Vérification des conteneurs Docker

```
0e:~/projects/ecf---Vite-Gourmand/vite-gourmand$ docker ps
```

7.6 Difficultés rencontrées lors du déploiement:

J'ai du faire plusieurs tests avant d'obtenir un environnement de déploiement qui fonctionne correctement.

La première difficulté rencontrée a été l'installation et la configuration de Docker Desktop sous Windows. La mise en place de WSL2 ainsi que la configuration de la virtualisation ont demandé plusieurs essais avant que les conteneurs puissent fonctionner correctement.

Une fois Docker opérationnel, j'ai modifié progressivement la structure du projet pour séparer chaque service dans son propre conteneur (Nginx, PHP, MySQL, MongoDB et Mailpit). Ça m'a permis de mieux comprendre comment fonctionne Docker et la manière dont les conteneurs communiquent entre eux.

Le déploiement sur le VPS ne s'est pas fait du premier coup. J'ai dû effectuer plusieurs ajustements sur la configuration de Nginx, les volumes Docker et la communication entre les différents services avant que tout fonctionne correctement.

Pendant le développement, j'ai aussi rencontré quelques difficultés avec Git et GitHub. J'ai donc revu l'organisation du dépôt afin de ne pas envoyer sur GitHub des fichiers inutiles, comme le dossier vendor, les fichiers d'environnement (.env) ou encore certains fichiers générés automatiquement.

7.7 Bilan du déploiement :

Le déploiement de Vite & Gourmand m'a permis de découvrir une nouvelle façon de préparer une application web avant sa mise en ligne.

Docker m'a permis de mieux comprendre comment fonctionne une application avec plusieurs services et pourquoi il est important d'avoir le même environnement en développement que sur le serveur.

Ce projet m'a permis d'apprendre à configurer un projet Symfony complet, à travailler avec MySQL et MongoDB, et à mettre en place une architecture Docker pour le développement et le déploiement.

Même si certaines étapes m'ont demandé beaucoup de recherches et plusieurs essais, elles m'ont permis de mieux comprendre le déploiement d'une application web et me serviront pour mes prochains projets.